

Phân tích độ phức tạp của thuật toán

Mục tiêu học tập

- Hiểu khái niệm về độ phức tạp của thuật toán (ký hiệu Big-O).
- Áp dụng các quy tắc để tính độ phức tạp: quy tắc hằng số, quy tắc cực đại, phép cộng, phép nhân.
- Xác định hoạt động chủ đạo (proxy tốt nhất) trong mã.
- Phân tích độ phức tạp về thời gian của các thuật toán đơn giản bằng các ví dụ Java.

Giải thuật và Cấu trúc dữ liệu

- Giải thuật (Algorithm): là tập hợp hữu hạn các bước được xác định rõ để giải quyết một bài toán.
- Cấu trúc dữ liệu (Data Structure): là cách tổ chức và lưu trữ dữ liệu sao cho có thể thao tác hiệu quả.
- Mối quan hệ: Giải thuật hoạt động trên cấu trúc dữ liệu để xử lý thông tin.

Giải thuật và Cấu trúc dữ liệu

Lập trình $\leftrightarrow$ Cấu trúc dữ liệu $\leftrightarrow$ Thuật toán

- Lập trình: công cụ biểu diễn
- Cấu trúc dữ liệu: cách tổ chức dữ liệu
- Thuật toán: phương pháp giải bài toán.

Ví dụ minh họa (Java)

Ví dụ: Tìm giá trị lớn nhất trong mảng:

```
int[] arr = {2, 5, 1, 9, 3};  
int max = arr[0];  
for (int i = 1; i < arr.length; i++) {  
    if (arr[i] > max)  
        max = arr[i];  
}  
System.out.println("Max = " + max);
```

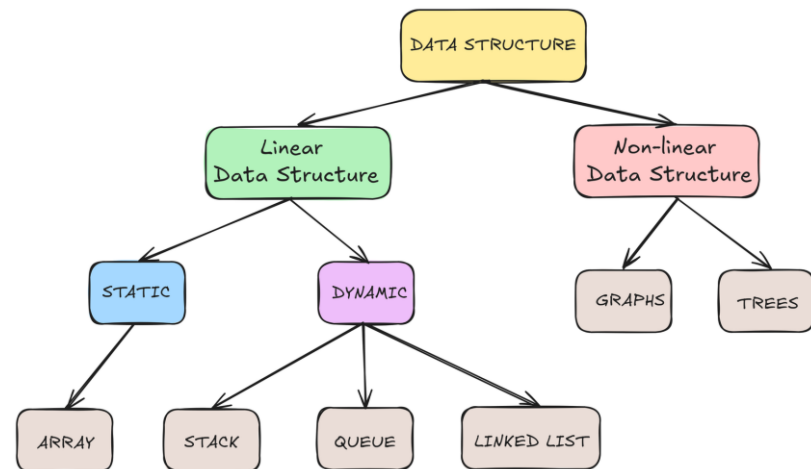
🤖 Câu hỏi: Nếu mảng có 1 triệu phần tử, thuật toán trên chạy trong bao lâu?

Phân loại cấu trúc dữ liệu

+ Tuyến tính: mảng, danh sách, ngăn xếp, hàng đợi.

+ Phi tuyến tính: cây, đồ thị.

Các tiêu chí lựa chọn cấu trúc dữ liệu: hiệu suất, độ phức tạp, khả năng mở rộng.



Ngôn ngữ diễn đạt giải thuật

- Giải thuật có thể được mô tả bằng:
 - + Ngôn ngữ tự nhiên
 - + Giả mã (pseudocode)
 - + Lưu đồ (flowchart)
 - + Ngôn ngữ lập trình (Java)
- Mục tiêu: Dễ hiểu, dễ kiểm chứng và dễ cài đặt.

Từ bài toán đến chương trình

- Các bước phát triển chương trình:
 1. Xác định bài toán
 2. Phân tích dữ liệu và yêu cầu
 3. Thiết kế giải thuật
 4. Cài đặt bằng ngôn ngữ lập trình
 5. Kiểm thử và tối ưu hóa.

Phân tích giải thuật – Đặt vấn đề

- Tại sao cần phân tích giải thuật?
 - + Đánh giá hiệu quả chương trình.
 - + So sánh giữa các phương án khác nhau.
- Hai tiêu chí chính: Thời gian (Time Complexity) và Bộ nhớ (Space Complexity).

Phân tích thời gian thực hiện giải thuật

- Độ phức tạp thời gian (Time Complexity): số phép toán cần thực hiện tùy theo kích thước đầu vào n .
- Độ phức tạp thời gian thể hiện sự tăng trưởng của thời gian chạy so với kích thước đầu vào n .
- Sử dụng ký hiệu Big-O: $O(1)$, $O(n)$, $O(n^2)$, $O(\log n)$... để mô tả giới hạn trên.
- Độ phức tạp giúp so sánh các thuật toán độc lập với phần cứng.

Ví dụ Java – Phân tích độ phức tạp

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        System.out.print(i + j);  
    }  
}
```

 Câu hỏi: Độ phức tạp thuật toán trên là bao nhiêu?

- A. $O(n)$
- B. $O(n^2)$
- C. $O(\log n)$
- D. $O(n \log n)$

Quy tắc xác định độ phức tạp

1. Quy tắc hằng số: loại bỏ các số nhân hằng số.
2. Quy tắc Max: $O(f+g) = O(\max(f,g))$.
3. Quy tắc cộng: các bộ phận tuần tự sẽ cộng thêm chi phí.
4. Quy tắc nhân: phép lặp lồng nhau sẽ nhân chi phí.

Quy tắc hằng số

- Nếu $T(n) = O(c * f(n))$ trong đó c là hằng số dương thì $T(n) = O(f(n))$.
- Ví dụ: $T(n) = O(5n) \Rightarrow O(n)$.

Quy tắc tối đa

- Nếu $T(n) = O(f(n) + g(n))$ thì $T(n) = O(\max(f(n), g(n)))$.
- Ví dụ: $O(n^2 + n) \Rightarrow O(n^2)$.

Quy tắc cộng

- Nếu $T1(n) = O(f(n))$ và $T2(n) = O(g(n))$, thì $T1 + T2 = O(f(n) + g(n))$.
- Chi phí mã tuần tự được cộng lại; giữ nguyên thuật ngữ chủ đạo.

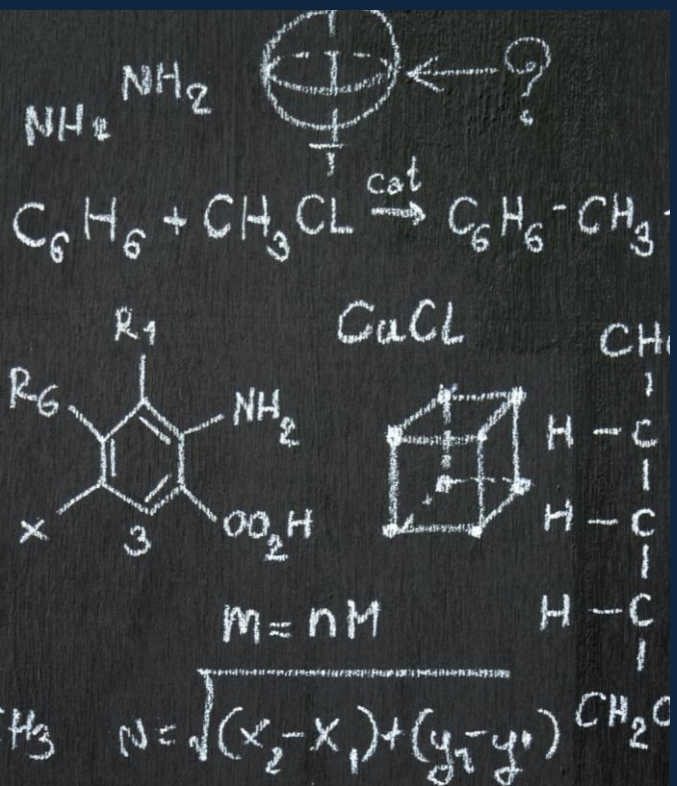
Quy tắc nhân

- Nếu một khối $O(f(n))$ được thực thi $k(n)=O(g(n))$ lần, tổng chi phí là $O(f(n)*g(n))$.
- Điều này giải thích sự phức tạp của các vòng lặp lồng nhau.

Hoạt động thống trị (Proxy tốt nhất)

- Xác định thao tác được thực hiện thường xuyên nhất (chiếm ưu thế).
- Đếm số lần phép toán chi phối được thực hiện theo hàm số của n .
- Sử dụng số đếm đó để xác định độ phức tạp tiệm cận.

Ví dụ 1: Vòng lặp tam giác lồng nhau



$s = 0;$

for ($i=0; i \leq n; i++$) {

$p = 1;$

for ($j=1; j \leq i; j++$)

$p = p * x / j;$

$s = s + p;$

}

Phép toán tích cực là $p = p * x / j$

Số lần thực hiện phép toán này là

$$0 + 1 + 2 + \dots + n = n(n-1)/2 = n^2/2 - n/2$$

=> Vậy độ phức tạp là $O(n^2/2 - n/2)$

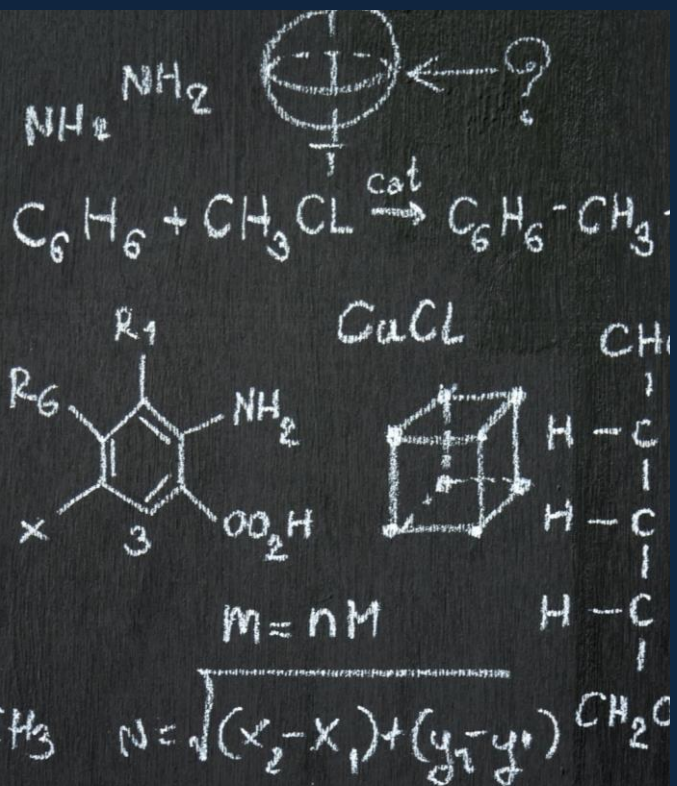
= $O(n^2/2)$ sử dụng quy tắc lấy max

= $O(n^2)$ sử dụng quy tắc bỏ hằng số

Ví dụ 2: Vòng lặp đơn

```
s = 1; p = 1;  
for (i=1;i<=n;i++) {  
    p = p * x / i;  
    s = s + p;  
}
```

Phép toán tích cực là $p = p * x / i$
Số lần thực hiện phép toán là n
 $\Rightarrow$ Vậy độ phức tạp của thuật toán là $O(n)$



Ví dụ 3

Thuật toán tính tổng các số từ 1 đến n

```
s=0;
```

```
for (i= 1;i<=n;i++)
```

```
    s=s+i;
```

Vòng lặp lặp n lần phép gán $s = s + i$, nên thời gian tính toán tỉ lệ thuận với n, tức độ phức tạp là $O(n)$.

=> độ phức tạp là $O(n)$

Ví dụ 4

```
for (i= 1;i<=n;i++)  
    for (j= 1;j<=n;j++)  
        //Lệnh
```

=> Dùng quy tắc nhân ta có $O(n^2)$
tương tự như vậy ta sẽ có $O(n^3)$, $O(n^4)$.

Ví dụ 5

```
for (i= 1;i<=n;i++)
```

```
    for (j= 1;j<=m;j++)
```

```
        //lệnh
```

=> Dùng quy tắc nhân ta có $O(n*m)$

Ví dụ 6

- for (i= 1;i<=n;i++)
 //lệnh 1
for (j= 1;j<=m;j++)
 //lệnh 2
=> Dùng quy tắc cộng và quy tắc lấy max,
sẽ có
- $O(\max(n,m))$

Ví dụ 7

```
for (i= 1;i<=n;i++) {  
    for (u= 1;u<=m;u++)  
        for (v= 1;v<=n;v++)  
            //lệnh  
    for j:= 1 to x do  
        for k:= 1 to z do  
            //lệnh  
}  
=> O(n*max (n*m,x*z))
```


Ví dụ 8

```
for (i= 1;i<=n;i++)  
    for (j= 1;j<=m;j++) {  
        for (k= 1;k<=x;k++)  
            //lệnh  
        for (h= 1;h<=y;h++)  
            //lệnh  
    }
```

=> $O(n*m* \max(x,y))$

Ví dụ 9

```
for (i= 1;i<=n;i++)  
    for (u= 1;u<= m;u++)  
        for (v= 1;v<=n;v++)  
            //lệnh  
  
for (j= 1;j<=x;j++)  
    for (k= 1;k<=z;k++)  
        //lệnh  
  
=>  $O(\max(m \cdot n^2, x \cdot z))$ 
```

Ví dụ 10

$$P(x) = x_m x^m + a_{m-1} x^{m-1} + \dots + a_1 x + a_0$$

$$Q(x) = b_n x^n + b_{n-1} x^{n-1} + \dots + b_1 x + b_0$$

if (m < n) p = m; else p = n;

for (i=0; i <= p; i++)

 c[i] = a[i] + b[i];

if (p < m)

 for (i=p+1; i <= m; i++) c[i] = a[i];

else

 for (i=p+1; i <= n; i++) c[i] = b[i];

while (p > 0 && c[p] == 0) p = p-1;

=> Độ phức tạp: $O(\max(m, n))$

Ví dụ 11

$$P(x) = x_m x^m + a_{m-1} x^{m-1} + \dots + a_1 x + a_0$$

$$Q(x) = b_n x^n + b_{n-1} x^{n-1} + \dots + b_1 x + b_0$$

$$p = m + n;$$

```
for (i=0; i<=p; i++) c[i] = 0;
```

```
for (i=0; i<=m; i++)
```

```
    for (j=0; j<=n; j++)
```

```
        c[i+j] = c[i+j] + a[i] + b[j];
```

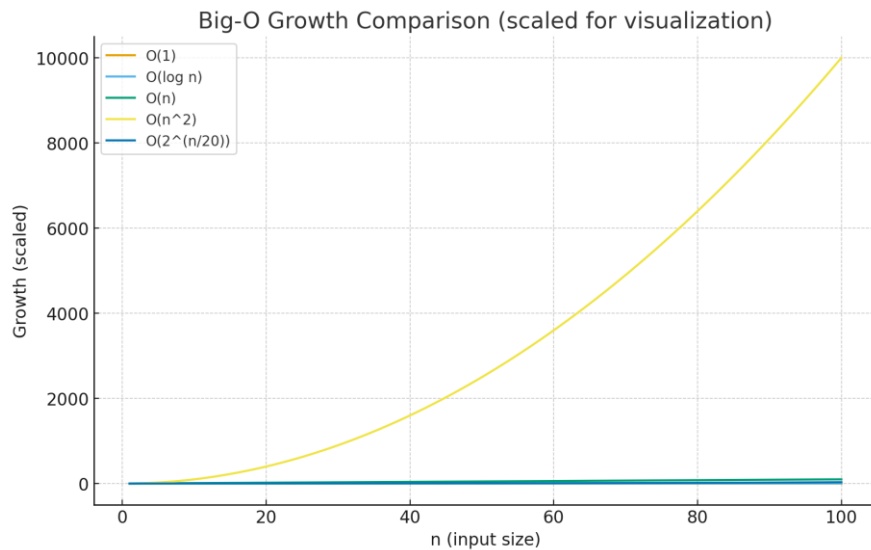
=> Độ phức tạp là $O(m.n)$

Ví dụ 12

- Phép nhân ngây thơ:
- với $i = 0$ đến m :
- đối với $j = 0$ đến n :
- $c[i+j] += a[i] * b[j]$;
- Độ phức tạp: $O(m * n)$.

Sắp xếp các độ phức tạp chung

- Độ phức tạp chung từ nhỏ nhất đến lớn nhất:
- $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, $O(n!)$
- Lưu ý: một số biểu thức trên cùng một 'hàng' là tương đương tiệm cận.



Biểu đồ tăng trưởng Big-O

Ghi chú so sánh trực quan

- Biểu đồ được thu nhỏ để dễ hình dung; tốc độ tăng theo cấp số nhân cực kỳ nhanh.
- Đối với n thực tế, $O(n \log n)$ thường tốt hơn $O(n^2)$.
- Big-O trừu tượng hóa các hằng số và các số hạng bậc thấp.

Bài tập

- 1) Phân tích độ phức tạp của vòng lặp lồng nhau:
`for(i=1;i<=n;i++) for(j=1;j<=i;j++) ...`
- 2) Nhân hai ma trận $n \times n$: độ phức tạp là bao nhiêu?
- 3) So sánh $O(n \log n)$ với $O(n^2)$ tại $n = 1000$ (định tính).

Bản tóm tắt

- Xác định hoạt động chi phối, đếm số lần thực hiện theo hàm số n .
- Áp dụng các quy tắc: hằng số, cực đại, phép cộng, phép nhân.
- Sử dụng Big-O để so sánh khả năng mở rộng của thuật toán.